A Number With a Name

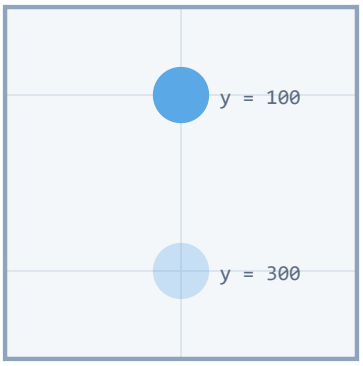
Making a variable

```
let y = 100;
```

`let` makes a new name. `y` is the name. `100` is what is in it.

Anywhere you could write a number, you can write the name instead.

```
circle(200, y, 60);
```



change the variable, not the shape

> **`let` goes at the top, outside `draw()`.** Inside `draw()` it is created fresh sixty times a second, so it can never grow — your sketch will sit perfectly still and the cause will be four lines away from where you are looking.

Making it move

```
function draw() {
  background(24, 38, 60);
  circle(200, y, 60);
  y = y + 1;
}
```

> **Read it as "becomes", not "equals".**
> `y = y + 1` means: take what is in `y`, add one, put it back. In math that sentence has no solution. Here it is an instruction, not a claim.

Why one line moves it more than once

`draw()` is not run once. It runs about **60 times every second**, forever, until you stop the sketch.

So one `y = y + 1` is sixty pixels of movement per second.

> **Speed is just how much you add.**
> `y = y + 1` slow · `y = y + 6` fast
> `y = y - 2` goes **up**, because y grows downward.

FRAME	Y
1	101
2	102
3	103
…	…
60	160

one second later

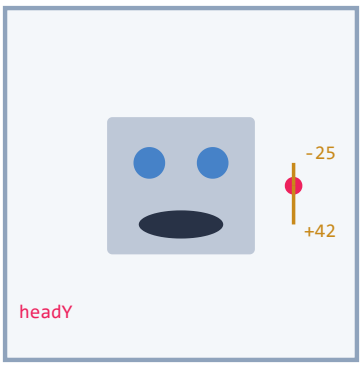
> `background()` is at the top of `draw()` for the same reason. Without it, every frame paints on top of the last one and a moving shape leaves a trail.

Moving more than one shape

A drawing is many shapes at many heights. If they all move together, each one has to keep its **distance** from the others.

Pick **one point**, put it in a variable, and write every other shape as an offset from it.

```
rect(200, headY, 160, 150);
circle(165, headY - 25, 34);
ellipse(200, headY + 42, 90, 30);
```



> **Working out an offset.** Subtract the starting value of the variable from where the shape currently sits. The window sat at `125` and `landerY` starts at `40`, so `125 - 40 = 85` — the window is `landerY + 85`.

> **The eyes are not at 165.** They are 25 above *wherever the head is*. Leave one shape on a fixed number and it stays behind while everything else moves — the head slides away and the face floats in mid-air.

Variables you did not make

NAME	WHAT IS IN IT
`width`	the canvas width, set by `createCanvas`
`height`	the canvas height
`mouseX`	where the cursor is across, right now
`mouseY`	where the cursor is down, right now

p5 updates `mouseX` and `mouseY` every frame. Some variables you change; some the system changes for you. Both are just names for numbers.

On the AP exam

What you write

```
let y = 100;
y = y + 1;
```

Exam Reference Sheet

```
y ← 100
y ← y + 1
```

> The AP exam is not written in JavaScript. It uses College Board's own notation, and the arrow `←` means **becomes** — the same word we use for `=`. No `let`, no semicolon.

Where this fits in AP CSP

Big Idea 3 — Algorithms and Programming · 30–35% of the AP Exam

CODE	LEARNING OBJECTIVE
`3.1`	Variables and Assignments
`AAP-1.A`	Represent a value with a variable
`AAP-1.B`	Determine the value of a variable as a result of an assignment

`3.A`	*Skill:* generalize data sources through variables
`4.B`	*Skill:* determine the result of code segments

> **Meaningful names matter to the exam, not just to you.** AAP-1.A.2 says variable names help readability. `landerY` says what it holds; `y` does not — and that is graded on the Create Performance Task.

Nothing you know yet can stop a falling shape. The variable keeps growing and the shape keeps going, off the screen and past it. To stop it, the program has to ask a question — *has it gone too far?* That is next lesson.